School of Computer Science and Artificial Intelligence

Lab Assignment # 08

Course Title : AI Assistant Coding
Name of Student : K. Srija
Enrollment No. : 2303A54023
Batch No. : 48

Lab 8: Test-Driven Development with AI – Generating and Working with Test Cases

Task Description #1 (Password Strength Validator – Apply AI in Security Context)
• Task: Apply AI to generate at least 3 assert test cases for is_strong_password(password) and implement the validator function.

• Requirements:
o Password must have at least 8 characters.
o Must include uppercase, lowercase, digit, and special character.
o Must not contain spaces.

Example Assert Test Cases:

```
assert is_strong_password("Abcd@123") == True
assert is_strong_password("abcd123") == False
assert is_strong_password("ABCD@1234") == True
```

Expected Output #1: • Password validation logic passing all AI-generated test cases.

Output Screenshot:

task1.py ● task2.py task3.py task4.py task5.py

LAB8.1.PY > task1.py > ...

```python
# Task 1: Password Strength Validator-TDD Style
# Step 1: Test Cases (TDD Style)

import re

def is_strong_password(password):
    if len(password) < 8:
        return False
    if " " in password:
        return False
    if not re.search(r"[A-Z]", password):
        return False
    if not re.search(r"[a-z]", password):
        return False
    if not re.search(r"[0-9]", password):
        return False
    if not re.search(r"[!@#$%^&*(),.?\":{}|<>]", password):
        return False
    return True


# Step 2: Test Cases (TDD Style)

def test_is_strong_password():
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
Password validation logic passing all AI-generated test cases.
PS C:\Users\saiva\OneDrive\Desktop\AI assisted coding> python -m doctest -v LAB8.1.PY/task1.py
Password validation logic passing all AI-generated test cases.
3 items had no tests:
    task1
    task1.is_strong_password
    task1.test_is_strong_password
0 tests in 3 items.
0 passed.
Test passed.
```

Explanation: This checks if a password is truly "strong" by making sure it has the right mix of characters and no spaces.

Task Description #2 (Number Classification with Loops – Apply AI for Edge Case Handling)

• Task: Use AI to generate at least 3 assert test cases for a classify_number(n) function. Implement using loops.

• Requirements:

- o Classify numbers as Positive, Negative, or Zero.
- o Handle invalid inputs like strings and None.
- o Include boundary conditions (-1, 0, 1).

Example Assert Test Cases:

```
assert classify_number(10) == "Positive"
assert classify_number(-5) == "Negative"
assert classify_number(0) == "Zero"
```

Expected Output #2: • Classification logic passing all assert tests.

Output Screenshot:

task1.py | task2.py | task3.py | task4.py | task5.py

LAB8.1.PY > task2.py > ...

```python
# Task 2: Number Classification
# Step 2: Implementation (Using Loop)

def classify_number(n):
    if not isinstance(n, (int, float)):
        return "Invalid Input"

    if n > 0:
        return "Positive"
    elif n < 0:
        return "Negative"
    else:
        return "Zero"


# Step 1: Test Cases (TDD Style)

def test_classify_number():
    # Positive number
    assert classify_number(10) == "Positive"

    # Negative number
    assert classify_number(-5) == "Negative"

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
Classification logic passing all AI-generated test cases.
PS C:\Users\saiva\OneDrive\Desktop\AI assisted coding> python -m doctest -v LAB8.1.PY/task2.py
Classification logic passing all AI-generated test cases.
3 items had no tests:
    task2
    task2.classify_number
    task2.test_classify_number
0 tests in 3 items.
0 passed.
Test passed.
```

Explanation: This sorts numbers into positive, negative, or zero, while politely rejecting anything that isn't a number.

Task Description #3 (Anagram Checker – Apply AI for String Analysis)

• Task: Use AI to generate at least 3 assert test cases form is_anagram(str1, str2) and implement the function.

• Requirements:

o Ignore case, spaces, and punctuation.

o Handle edge cases (empty strings, identical words).

Example Assert Test Cases:

assert is_anagram("listen", "silent") == True

assert is_anagram("hello", "world") == False

assert is_anagram("Dormitory", "Dirty Room") == True

Expected Output #3: • Function correctly identifying anagrams and passing all AI-generated tests.

Output Screenshot:

task1.py task2.py task3.py task4.py task5.py

LAB8.1.PY > task4.py > ...

```python
# Task 4: Inventory Class
# Step 1: Test Cases (TDD Style)

def test_inventory():
    inv = Inventory()

    # Add item
    inv.add_item("Pen", 10)
    assert inv.get_stock("Pen") == 10

    # Remove item
    inv.remove_item("Pen", 5)
    assert inv.get_stock("Pen") == 5

    # Add another item
    inv.add_item("Book", 3)
    assert inv.get_stock("Book") == 3

    # Removing more than available should not go below 0
    inv.remove_item("Pen", 10)
    assert inv.get_stock("Pen") == 0

    # Non-existing item should return 0
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\saiva\OneDrive\Desktop\AI assisted coding> python -m doctest -v LAB8.1.PY/task4.py
Inventory system passing all AI-generated test cases.
7 items had no tests:
    task4
    task4.Inventory
    task4.Inventory.__init__
    task4.Inventory.add_item
    task4.Inventory.get_stock
    task4.Inventory.remove_item
    task4.test_inventory
0 tests in 7 items.
0 passed.
```

Explanation: This spots whether two words or phrases are made of the same letters, ignoring case, spaces, and punctuation.

Task Description #4 (Inventory Class – Apply AI to Simulate Real- World Inventory System)

• Task: Ask AI to generate at least 3 assert-based tests for an Inventory class with stock management.

• Methods:

- o add_item(name, quantity)
- o remove_item(name, quantity)
- o get_stock(name)

Example Assert Test Cases:

```
inv = Inventory()
inv.add_item("Pen", 10)
assert inv.get_stock("Pen") == 10
inv.remove_item("Pen", 5)
assert inv.get_stock("Pen") == 5
```

inv.add_item("Book", 3)
assert inv.get_stock("Book") == 3

Expected Output #4: • Fully functional class passing all assertions.

Output Screenshot:

task1.py task2.py task3.py task4.py task5.py

LAB8.1.PY > task4.py > ...

```python
# Task 4: Inventory Class
# Step 1: Test Cases (TDD Style)

def test_inventory():
    inv = Inventory()

    # Add item
    inv.add_item("Pen", 10)
    assert inv.get_stock("Pen") == 10

    # Remove item
    inv.remove_item("Pen", 5)
    assert inv.get_stock("Pen") == 5

    # Add another item
    inv.add_item("Book", 3)
    assert inv.get_stock("Book") == 3

    # Removing more than available should not go below 0
    inv.remove_item("Pen", 10)
    assert inv.get_stock("Pen") == 0

    # Non-existing item should return 0
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\saiva\OneDrive\Desktop\AI assisted coding> python -m doctest -v LAB8.1.PY/task4.py
Inventory system passing all AI-generated test cases.
7 items had no tests:
    task4
    task4.Inventory
    task4.Inventory.__init__
    task4.Inventory.add_item
    task4.Inventory.get_stock
    task4.Inventory.remove_item
    task4.test_inventory
0 tests in 7 items.
0 passed.
```

Explanation: This simulates a mini store system where you can add, remove, and check stock for items.

Task Description #5 (Date Validation & Formatting – Apply AI for Data Validation)

• Task: Use AI to generate at least 3 assert test cases for validate_and_format_date(date_str) to check and convert dates.

• Requirements:

o Validate "MM/DD/YYYY" format.
o Handle invalid dates.
o Convert valid dates to "YYYY-MM-DD".

Example Assert Test Cases:

assert validate_and_format_date("10/15/2023") == "2023-10-15"

assert validate_and_format_date("02/30/2023") == "Invalid Date"
assert validate_and_format_date("01/01/2024") == "2024-01-01"

Expected Output #5: • Function passes all AI-generated assertions and handles edge cases.

Output Screenshot:

task1.py task2.py task3.py task4.py task5.py

LAB8.1.PY > task5.py > ...

```python
# Task 5: Date Validation & Formatting

# Step 1: Test Cases (TDD Style)

def test_validate_and_format_date():
    # Valid dates
    assert validate_and_format_date("10/15/2023") == "2023-10-15"
    assert validate_and_format_date("01/01/2024") == "2024-01-01"

    # Invalid date (February 30 does not exist)
    assert validate_and_format_date("02/30/2023") == "Invalid Date"

    # Invalid month
    assert validate_and_format_date("13/01/2023") == "Invalid Date"

    # Wrong format / non-date string
    assert validate_and_format_date("abc") == "Invalid Date"


# Step 2: Implementation

from datetime import datetime
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
d coding\LAB8.1.PY\task5.py'
Date validation logic passing all AI-generated test cases.
PS C:\Users\saiva\OneDrive\Desktop\AI assisted coding> python -m doctest -v LAB8.1.PY/task5.py
Date validation logic passing all AI-generated test cases.
3 items had no tests:
    task5
    task5.test_validate_and_format_date
    task5.validate_and_format_date
0 tests in 3 items.
0 passed.
Test passed.
```

Explanation: This ensures a date is valid in “MM/DD/YYYY” format and neatly converts it into “YYYY-MM-DD.